

The University of Azad Jammu & Kashmir, Muzaffarabad



DATABASEMANAGEMENTSYSTEM • LAB

Open-Ended Lab Report

Car Rental Management System

CarGo Rentals · MySQL 8 (InnoDB) Implementation

Name: Shahzad Ahmed Awan

Roll no: 2024-SE-15

Course Title: Database Systems

Submitted to: Awais Rathore

Submission Date: 21th September, 2026

DBMS — OPEN-ENDED LAB

1. INTRODUCTION

The CarGo Rentals system was designed to manage customers, vehicles, rentals, and payments using a structured relational database.

The database replaces manual record handling with a normalized system that supports accurate rental management, reporting, automation, and efficient data retrieval.

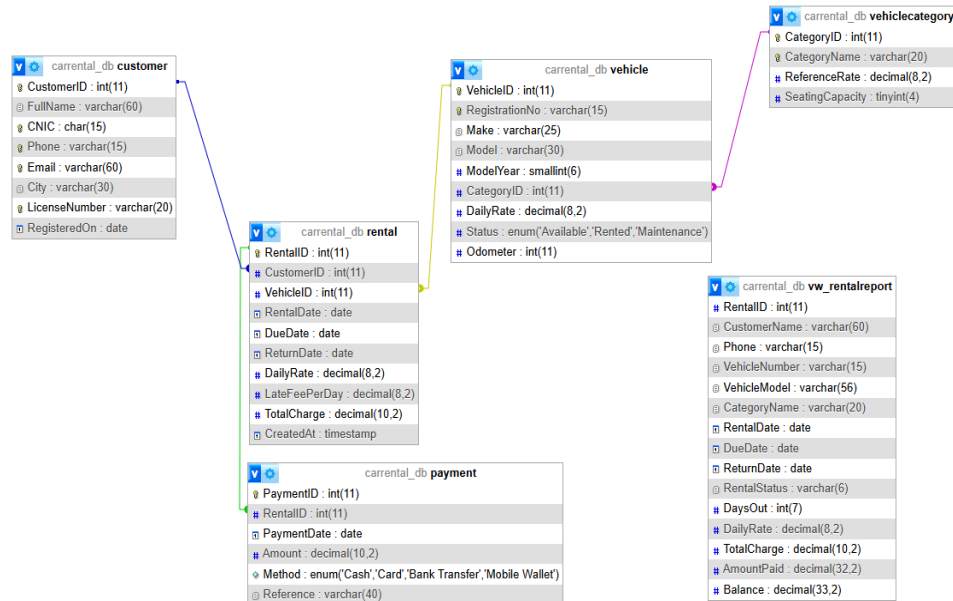
OBJECTIVES

- Design a relational database for CarGo Rentals.
- Apply normalization up to 3NF.
- Implement tables, keys, constraints, and sample data.
- Use JOINS for operational reports.
- Implement a VIEW, TRIGGER, and STORED PROCEDURE.
- Improve query performance through optimization.

MAIN BUSINESS RULES

- Each customer has a unique CNIC.
- Vehicles are organized into categories with standard daily rates.
- A vehicle can have multiple rental records over time.
- A vehicle should not be available for another rental while rented.
- Payments are associated with rentals and may be recorded separately.

2. DATABASE DESIGN



The system consists of five main entities:

Table	Purpose
Customer	Stores customer information
VehicleCategory	Stores vehicle categories and daily rates
Vehicle	Stores fleet and vehicle status
Rental	Records customer-vehicle rentals
Payment	Stores rental payment records

KEY RELATIONSHIPS

- One category can contain multiple vehicles.
- One customer can have multiple rentals.
- One vehicle can have multiple rental records over time.
- One rental can have multiple payment records.

3. Normalization

The initial rental data is maintained in a single relation combining customer, vehicle, rental, and payment details. The database is normalized step-by-step from Unnormalized Form (UNF) through First Normal Form (1NF), Second Normal Form (2NF), and Third Normal Form (3NF) to eliminate data redundancy and anomalies.

3.1 Unnormalized Form (UNF)

Explanation:

In UNF, data from multiple functional entities is combined into a single flat structure. Storing non-atomic or multi-valued fields (such as combined payment transactions) inside a single record creates heavy data redundancy and operational anomalies.

Attributes:

RentalID, CustomerName, CustomerPhone, VehicleNumber, VehicleModel, DailyRate, RentalDate, ReturnDate, PaymentAmount

RentalID	CustomerName	CustomerPhone	VehicleNumber	VehicleModel	DailyRate	RentalDate	ReturnDate	PaymentAmount
R001	Ali Khan	0300-1234567	ABC-123	Toyota Corolla	5000.00	2026-09-01	2026-09-04	10000, 5000

Figure 3.1: Initial rental relation in Unnormalized Form (UNF).

3.2 First Normal Form (1NF)

Explanation:

A table is in 1NF when every column contains atomic (indivisible) values and there are no repeating groups or multi-valued entries. Non-atomic payment strings are converted into a single atomic total amount per record.

Transformation: Rental_UNF → Rental_1NF

RentalID	CustomerName	CustomerPhone	VehicleNumber	VehicleModel	DailyRate	RentalDate	ReturnDate	PaymentAmount
R001	Ali Khan	0300-1234567	ABC-123	Toyota Corolla	5000.00	2026-09-01	2026-09-04	15000.00

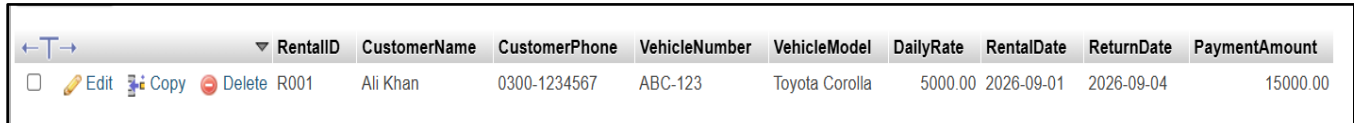
Figure 3.2: Rental relation after conversion to First Normal Form (1NF).

3.3 Second Normal Form (2NF)

Explanation:

A relation is in 2NF if it is in 1NF and all non-key attributes are fully dependent on the primary key (no partial dependencies). Since RentalID serves as a single-attribute primary key, partial dependency cannot occur. The schema satisfies 2NF without structural decomposition.

Transformation: Rental_1NF $\rightarrow$ Rental_2NF



	RentalID	CustomerName	CustomerPhone	VehicleNumber	VehicleModel	DailyRate	RentalDate	ReturnDate	PaymentAmount
	R001	Ali Khan	0300-1234567	ABC-123	Toyota Corolla	5000.00	2026-09-01	2026-09-04	15000.00

Figure 3.3: Rental relation satisfying Second Normal Form (2NF).

3.4 Third Normal Form (3NF)

Explanation:

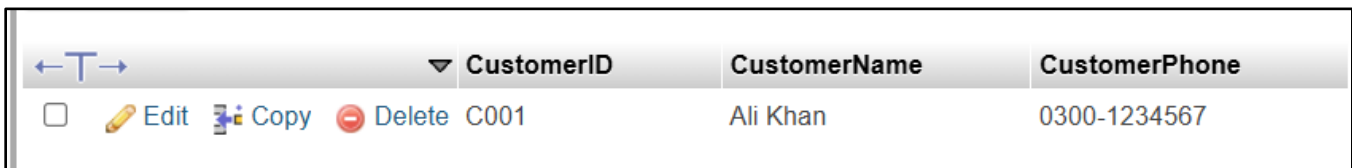
A relation is in 3NF when it is in 2NF and contains no transitive dependencies (where a non-key attribute determines another non-key attribute).

Transitive dependencies present in 2NF:

- CustomerID $\rightarrow$ CustomerName, CustomerPhone
- VehicleNumber $\rightarrow$ VehicleModel, CategoryID
- CategoryID $\rightarrow$ DailyRate (Rates depend on vehicle category)

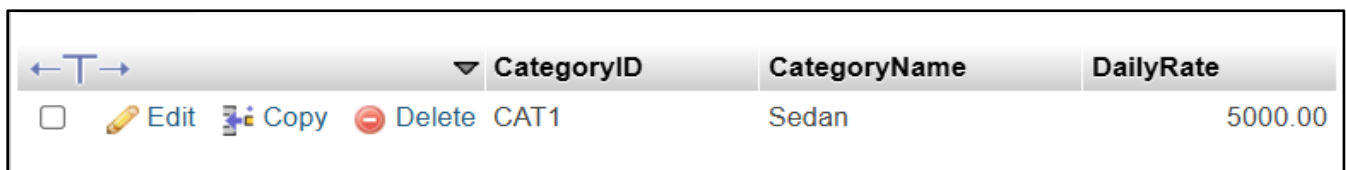
To resolve these, surrogate keys (CustomerID, CategoryID) are introduced, and the relation is decomposed into five distinct entities.

Customer:



	CustomerID	CustomerName	CustomerPhone
	C001	Ali Khan	0300-1234567

VehicleCategory:



	CategoryID	CategoryName	DailyRate
	CAT1	Sedan	5000.00

Vehicle:

				VehicleNumber	VehicleModel	CategoryID
				ABC-123	Toyota Corolla	CAT1

Rental:

				RentalID	CustomerID	VehicleNumber	RentalDate	ReturnDate
				R001	C001	ABC-123	2026-09-01	2026-09-04

Payment:

				PaymentID	RentalID	PaymentAmount
				1	R001	15000.00

3.5 Normalization Summary

Normal Form	Applied Transformation	Resolved Problem / Anomaly
UNF	Combined customer, vehicle, rental, and payment fields into a single relation.	Represents the raw spreadsheet dataset.
1NF	Converted multi-valued attributes into single atomic values.	Eliminates non-atomic storage issues.
2NF	Verified single-attribute primary key (RentalID).	Confirms no partial dependencies exist.
3NF	Separated relation into Customer, VehicleCategory, Vehicle, Rental, and Payment tables.	Eliminates transitive dependencies, pricing redundancies, and update anomalies.

4. DATABASE IMPLEMENTATION

The database CarRentalDB was implemented using five related tables:

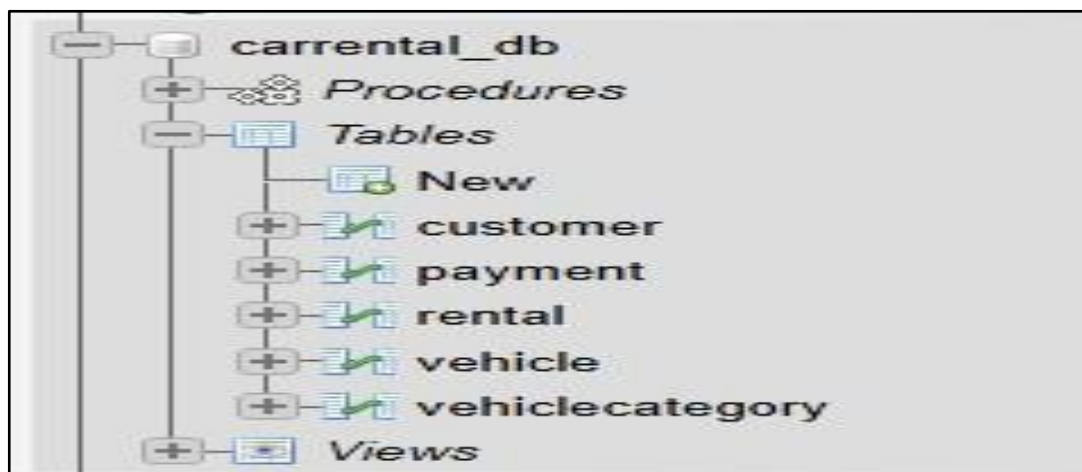
- Customer
- VehicleCategory
- Vehicle
- Rental
- Payment

Primary keys identify individual records, while foreign keys maintain relationships between related tables. UNIQUE, NOT NULL and DEFAULT constraints were also applied where required.



Sample records were inserted to represent realistic customers, vehicles, categories, rentals and payments.

Figure 2. Implemented database structure



5. JOIN QUERIES AND RESULTS

The required JOIN operations were implemented to extract useful rental information from multiple related tables. The lab manual requires reporting for rentals, customers, vehicles, and rental counts.

5.1 ACTIVE RENTAL REPORT

RentalID	CustomerName	VehicleNumber	VehicleModel	RentalDate	ReturnDate
1	Ali Khan	LEB-4521	Corolla	2026-06-01	2026-06-04
5	Ali Khan	ISB-1145	Alto	2026-07-08	2026-07-10
10	Ali Khan	LEA-7788	City	2026-09-05	2026-09-08
2	Sara Iqbal	KHI-3092	Cultus	2026-06-10	2026-06-13
8	Sara Iqbal	LEB-2210	Civic	2026-08-05	2026-08-09
3	Bilal Ahmed	LEA-7788	City	2026-06-15	2026-06-20
9	Bilal Ahmed	LEB-4521	Corolla	2026-08-12	2026-08-16
4	Fatima Sheikh	LEC-9001	Fortuner	2026-07-01	2026-07-05
11	Fatima Sheikh	LEC-9001	Fortuner	2026-09-16	NULL
6	Usman Tariq	ISB-8834	Hiace	2026-07-15	2026-07-24
13	Usman Tariq	KHI-6612	Sportage	2026-09-18	NULL
7	Ayesha Noor	KHI-6612	Sportage	2026-08-01	2026-08-04
12	Ayesha Noor	LEB-2210	Civic	2026-09-17	NULL

Displays the customer, vehicle, category/rate information and rental date for currently active rentals.

5.2 CUSTOMERS AND THEIR RENTALS

Displays customers together with their rented vehicles, including customers who currently have no rental.

CustomerID	CustomerName	RentalID	VehicleNumber	RentalDate	ReturnDate
1	Ali Khan	1	LEB-4521	2026-06-01	2026-06-04
1	Ali Khan	5	ISB-1145	2026-07-08	2026-07-10
1	Ali Khan	10	LEA-7788	2026-09-05	2026-09-08
2	Sara Iqbal	2	KHI-3092	2026-06-10	2026-06-13
2	Sara Iqbal	8	LEB-2210	2026-08-05	2026-08-09
3	Bilal Ahmed	3	LEA-7788	2026-06-15	2026-06-20
3	Bilal Ahmed	9	LEB-4521	2026-08-12	2026-08-16
4	Fatima Sheikh	4	LEC-9001	2026-07-01	2026-07-05
4	Fatima Sheikh	11	LEC-9001	2026-09-16	NULL
5	Usman Tariq	6	ISB-8834	2026-07-15	2026-07-24
5	Usman Tariq	13	KHI-6612	2026-09-18	NULL
6	Ayesha Noor	7	KHI-6612	2026-08-01	2026-08-04
6	Ayesha Noor	12	LEB-2210	2026-09-17	NULL
7	Zain Abbas	NULL	NULL	NULL	NULL
8	Hira Yousaf	NULL	NULL	NULL	NULL

5.3 VEHICLE RENTAL STATUS

Displays all vehicles and their current rental information, including vehicles that are currently unrented.

VehicleID	1	VehicleNumber	VehicleModel	CustomerName	RentalDate	DueDate
	1	LEB-4521	Corolla	NULL	NULL	NULL
	2	LEA-7788	City	NULL	NULL	NULL
	3	KHI-3092	Cultus	NULL	NULL	NULL
	4	ISB-1145	Alto	NULL	NULL	NULL
	5	LEC-9001	Fortuner	Fatima Sheikh	2026-09-16	2026-09-20
	6	KHI-6612	Sportage	Usman Tariq	2026-09-18	2026-09-25
	7	ISB-8834	Hiace	NULL	NULL	NULL
	8	LEB-2210	Civic	Ayesha Noor	2026-09-17	2026-09-19
	9	KHI-4455	Wagon R	NULL	NULL	NULL
	10	ISB-3377	Land Cruiser	NULL	NULL	NULL

5.4 TOTAL RENTALS BY CUSTOMER

Counts the number of rentals for each customer, including customers with zero rentals.

CustomerID	1	CustomerName	TotalRentals	TotalRevenue
	1	Ali Khan	3	46500.00
	2	Sara Iqbal	2	47000.00
	3	Bilal Ahmed	2	57625.00
	4	Fatima Sheikh	2	56000.00
	5	Usman Tariq	2	114000.00
	6	Ayesha Noor	2	33000.00
	7	Zain Abbas	0	0.00
	8	Hira Yousaf	0	0.00

5.5 REVENUE ANALYSIS

An additional query was used to summarize payment revenue by customer.

CustomerID	CustomerName	TotalRevenue
1	Ali Khan	46500.00
2	Sara Iqbal	47000.00
3	Bilal Ahmed	57625.00
4	Fatima Sheikh	86000.00
5	Usman Tariq	100000.00
6	Ayesha Noor	33000.00
7	Zain Abbas	0.00
8	Hira Yousaf	0.00

6. VIEW

A database VIEW was created to provide a consolidated list of currently available vehicles along with their category and daily rental rate.

Purpose

The view simplifies repeated fleet-availability queries and provides a reusable source for operational reporting.

Figure 9. Available fleet VIEW

	VehicleID	VehicleNumber	Make	Model	CategoryName	DailyRate	Status
☐ Edit Copy Delete	3	KHI-3092	Suzuki	Cultus	Economy	4000.00	Available
☐ Edit Copy Delete	4	ISB-1145	Suzuki	Alto	Economy	4500.00	Available
☐ Edit Copy Delete	1	LEB-4521	Toyota	Corolla	Sedan	6500.00	Available
☐ Edit Copy Delete	2	LEA-7788	Honda	City	Sedan	6000.00	Available
☐ Edit Copy Delete	10	ISB-3377	Toyota	Land Cruiser	SUV	25000.00	Available
☐ Edit Copy Delete	7	ISB-8834	Toyota	Hiace	Van	12000.00	Available

7. TRIGGER

Four triggers were implemented on the `Rental` table to enforce the business rule that a vehicle cannot be double-booked or rented while under maintenance, and to keep `Vehicle.Status` synchronized automatically.



The screenshot shows a database management interface with a table of triggers for the `carrental_db` database. The table has columns: Trigger, Event, Table, Statement, Timing, Created, sql_mode, Definer, character_set_client, collation_connection, and Database Collation. There are four triggers listed:

Trigger	Event	Table	Statement	Timing	Created	sql_mode	Definer	character_set_client	collation_connection	Database Collation
trg_rental_before_insert	INSERT	rental	BEGIN DECLARE v_open_count INT; DECLARE ...	BEFORE	2026-09-26 16:33:44.06	NO_ZERO_IN_DATE,NO_ZERO_DATE,NO_ENGINE_SUBSTITUTIO...	root@localhost	utf8mb4	utf8mb4_unicode_ci	utf8mb4_general_ci
trg_rental_after_insert	INSERT	rental	BEGIN IF NEW.ReturnDate IS NULL THEN	AFTER	2026-09-26 16:33:44.08	NO_ZERO_IN_DATE,NO_ZERO_DATE,NO_ENGINE_SUBSTITUTIO...	root@localhost	utf8mb4	utf8mb4_unicode_ci	utf8mb4_general_ci
trg_rental_before_update	UPDATE	rental	BEGIN IF OLD.VehicleID <> NEW.VehicleID THEN	BEFORE	2026-09-26 16:33:44.07	NO_ZERO_IN_DATE,NO_ZERO_DATE,NO_ENGINE_SUBSTITUTIO...	root@localhost	utf8mb4	utf8mb4_unicode_ci	utf8mb4_general_ci
trg_rental_after_update	UPDATE	rental	BEGIN IF OLD.ReturnDate IS NULL AND NEW.Return...	AFTER	2026-09-26 16:33:44.10	NO_ZERO_IN_DATE,NO_ZERO_DATE,NO_ENGINE_SUBSTITUTIO...	root@localhost	utf8mb4	utf8mb4_unicode_ci	utf8mb4_general_ci

7.1 trg_rental_before_insert

Fires before a new rental row is inserted.

- If the vehicle already has an open rental (`ReturnDate IS NULL`) → insert is blocked (SQLSTATE '45000').
- If the vehicle's status is 'Maintenance' → insert is blocked as well.

Error

SQL query: [Copy](#)

```
INSERT INTO Rental (CustomerID, VehicleID, RentalDate, DueDate, DailyRate)
VALUES (7, 5, '2026-09-19', '2026-09-21', 14000.00);
```

MySQL said: 

#1644 - Vehicle is already on an open rental.

7.2 trg_rental_before_update

Prevents the vehicle assigned to a rental from being changed after creation, protecting the integrity of historical records.

Error

SQL query: [Copy](#)

```
INSERT INTO Rental (CustomerID, VehicleID, RentalDate, DueDate, DailyRate)
VALUES (7, 9, '2026-09-19', '2026-09-21', 4200.00);
```

MySQL said: 

```
#1644 - Vehicle is under maintenance and cannot be rented.
```

7.3 trg_rental_after_insert

After a new open rental is inserted, automatically sets `Vehicle.Status = 'Rented'`.

✓ MySQL returned an empty result set (i.e. zero rows). (Query took 0.0037 seconds.)

```
CALL sp_RegisterRental(7, 1, '2026-09-19', '2026-09-22', @new_rental, @estimate);
```

[\[Edit inline \]](#) [\[Edit \]](#) [\[Create PHP code \]](#)

⚠ Current selection does not contain a unique column. Grid edit, checkbox, Edit, Copy and Delete features are not available. 

✓ Showing rows 0 - 0 (1 total, Query took 0.0002 seconds.)

```
SELECT @new_rental AS NewRentalID, @estimate AS EstimatedCharge;
```

☐ Profiling [\[Edit inline \]](#) [\[Edit \]](#) [\[Explain SQL \]](#) [\[Create PHP code \]](#) [\[Refresh \]](#)

☐ Show all | Number of rows: 25  | Filter rows:

Extra options

NewRentalID	EstimatedCharge
15	19500.00

7.4 trg_rental_after_update

When a rental's `ReturnDate` is filled in (the car is returned), automatically sets `Vehicle.Status = 'Available'`.

Purpose: Together, these triggers make the "one vehicle, one active rental at a time" rule impossible to violate at the application layer, and remove the need to manually update vehicle status on every rental or return.

8. STORED PROCEDURE

Two stored procedures automate the two main rental transactions.

Db	Name	Type	Definer	Modified	Created	Security_type	Comment	character_set_client	collation_connection	Database Collation
carrental_db	sp_CloseRental	PROCEDURE	root@localhost	2026-09-26 16:50:05	2026-09-26 16:50:05	DEFINER		utf8mb4	utf8mb4_unicode_ci	utf8mb4_general_ci
carrental_db	sp_RegisterRental	PROCEDURE	root@localhost	2026-09-26 16:50:05	2026-09-26 16:50:05	DEFINER		utf8mb4	utf8mb4_unicode_ci	utf8mb4_general_ci

8.1 sp_RegisterRental

IN: CustomerID, VehicleID, RentalDate, DueDate → OUT: RentalID, EstimatedCharge

- Validates the customer and vehicle exist.
- Validates the due date is not before the rental date.
- Pulls the vehicle's current daily rate.
- Calculates the estimated charge.
- Freezes a late-fee rate (25% of daily rate) into the rental record so future rate changes don't affect existing contracts.

- Inserts the new rental (validated by the Section 7 triggers).

✓ MySQL returned an empty result set (i.e. zero rows). (Query took 0.0072 seconds.)

```
CALL sp_RegisterRental(7, 1, '2026-09-19', '2026-09-22', @new_rental, @estimate);
```

[Edit inline] [Edit] [Create PHP code]

⚠ Current selection does not contain a unique column. Grid edit, checkbox, Edit, Copy and Delete features are not available.

✓ Showing rows 0 - 0 (1 total, Query took 0.0002 seconds.)

```
SELECT @new_rental AS NewRentalID, @estimate AS EstimatedCharge;
```

☐ Profiling [Edit inline] [Edit] [Explain SQL] [Create PHP code] [Refresh]

☐ Show all | Number of rows: 25 | Filter rows: Search this table

Extra options

NewRentalID	EstimatedCharge
15	19500.00

8.2 sp_CloseRental

IN: RentalID, ReturnDate, PaymentMethod

- Validates the rental exists and isn't already closed.
- Calculates chargeable days and any late days past the due date.
- Computes the total charge.
- Updates the rental with the return date and total (fires trg_rental_after_update, freeing the vehicle).

✓ Showing rows 0 - 0 (1 total, Query took 0.0052 seconds.)

```
CALL sp_CloseRental(11, '2026-09-23', 'Cash');
```

[Edit inline] [Edit] [Create PHP code]

☐ Show all | Number of rows: 25 | Filter rows: Search this table

Extra options

RentalID	ChargeableDays	LateDays	DailyRate	LateFeePerDay	TotalCharge	AlreadyPaid	CollectedNow
11	7	3	14000.00	3500.00	108500.00	30000.00	78500.00

- Compares the total against payments already received and auto-inserts a settlement payment for any balance.
- Returns a one-row receipt summary: days, late days, rate, total, already paid, collected now.

Purpose: These two procedures encapsulate the full rental lifecycle behind a single call each, so the charge and late-fee math is calculated consistently rather than re-implemented in application code.

9. QUERY OPTIMIZATION

9.1 Problem identified

The Rental table had no indexes beyond its primary key. Two operations hit it repeatedly:

- Checking whether a vehicle has an open rental (runs inside `trg_rental_before_insert` on *every* booking).
- Filtering rentals by date range for management reports.

9.2 Impact

Without a supporting index, both operations force a full table scan. This is invisible with a handful of rows, but the availability check runs on the hottest write path in the system, so the cost grows with every booking as the table scales.

9.3 Improvement

- `idx_rental_vehicle_open` (VehicleID, ReturnDate) — supports the availability check.
- `idx_rental_date` (RentalDate) — supports date-range reporting.

10. RESULTS AND DISCUSSION

The implemented database provides a structured solution for managing the main operations of CarGo Rentals.

The final system supports:

- Normalized customer, vehicle, rental and payment data.
- Multi-table reporting through JOINS.

- Reusable reporting through a VIEW.
- Automatic vehicle-status updates through a TRIGGER.
- Automated return processing through a STORED PROCEDURE.
- Improved query performance through optimization.

The implemented components satisfy the major database design, normalization, query, advanced database object, and optimization requirements of the lab.

11. CONCLUSION

The CarGo Rentals database was successfully designed and implemented as a normalized relational database. The system manages core rental operations while maintaining data consistency and providing useful reports.

JOIN queries, a VIEW, a TRIGGER, a STORED PROCEDURE, and query optimization were incorporated to demonstrate practical DBMS concepts in a real-world rental management scenario.